

Son güncelleme : 09/2026

İçindekiler

- Git Nedir?
- Git Yapılandırma
- Git Yapılandırma

Versiyon Kontrol Sistemi (Version Control System — VCS)

Bir proje geliştirirken yaptığın her değişikliğin ne zaman, kim tarafından, neden yapıldığını kaydetmek ve istediğin ana geri dönmek için kullanılır. Dosyalardaki değişikliklerin geçmişini takip eden ve geri dönüş imkânı veren bir sistemdir.

VCS'nin Kısa Tarihi

🏛️ 1. Nesil: Yerel Sistemler (1970-80'ler)

İlk VCS'ler tek bir bilgisayarda çalışıyordu.

RCS (Revision Control System, 1982): Dosyaların değişiklik geçmişini yerel olarak saklıyordu. Tek bir kullanıcı, tek bir bilgisayar.

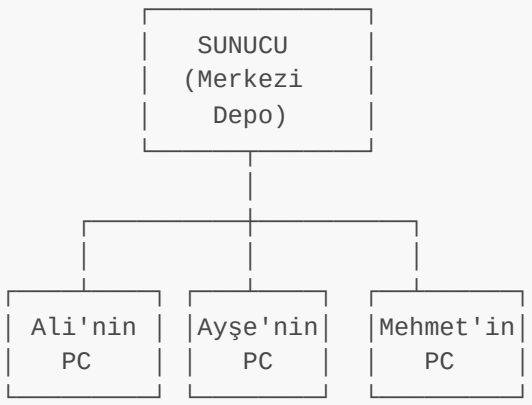
```
┌──────────────────────────────┐
│         Bilgisayarım         │
│                              │
│  dosya.c  ⟷  RCS veritabanı  │
│              (değişiklikler) │
└──────────────────────────────┘
```

> **Sorun:** Ekip çalışması yok. Herkes kendi makinesinde. Dosyaları paylaşmak için USB bellek veya e-posta gerekiyor.

🏢 2. Nesil: Merkezi Sistemler (1990-2000'ler)

CVS (1990) ve Subversion/SVN (2000) ile merkezi model.

Tek bir sunucu ile herkes bu sunucuya bağlanıyor. Değişiklikleri sunucuya gönderiyor, sunucudan alıyor.




```
Her PC'de sadece dosyaların SON HALİ var.
Geçmiş? Sadece sunucuda.
```

> **İyileştirme:** Artık ekip çalışması mümkün. Ali ve Ayşe aynı projede çalışabiliyor.

Merkezi (Centralized) Model Dezavantajları:

- **Tek nokta arızası (Single Point of Failure):** Sunucu çökerse? Kimse çalışamaz. Geçmiş kaybolabilir.
- **Ağ bağımlılığı:** İnternetsiz commit bile yapamazsın.
- **Yavaşlık:** Her işlem sunucuyla iletişim gerektirdiği için log bakmak bile ağ hızına bağlıydı.
- **Branching:** SVN'de branch açmak çok yavaş ve karmaşıktı.

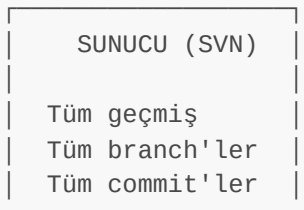
🌐 3. Nesil: Dağıtık Sistemler (2005-Günümüz)

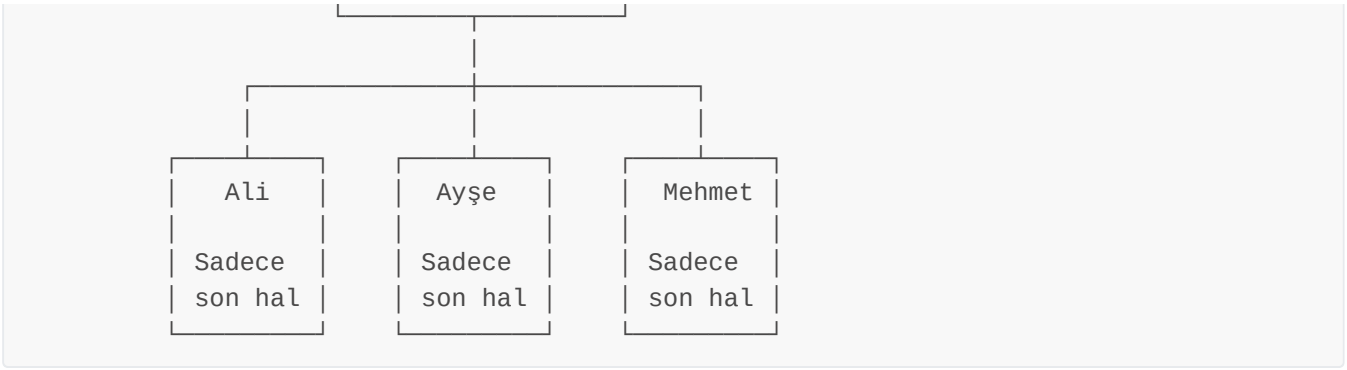
Dağıtık (Distributed) Model — Git ve Türevleri

- Her geliştirici, projenin tam kopyasına sahip.
- İnternetsiz çalışabilirsin.
- Sunucu çökerse? Herkesin yedeği var. Diğer bilgisayardan komple geri yükleme yapılabilir.

Merkezi vs Dağıtık: Temel Fark

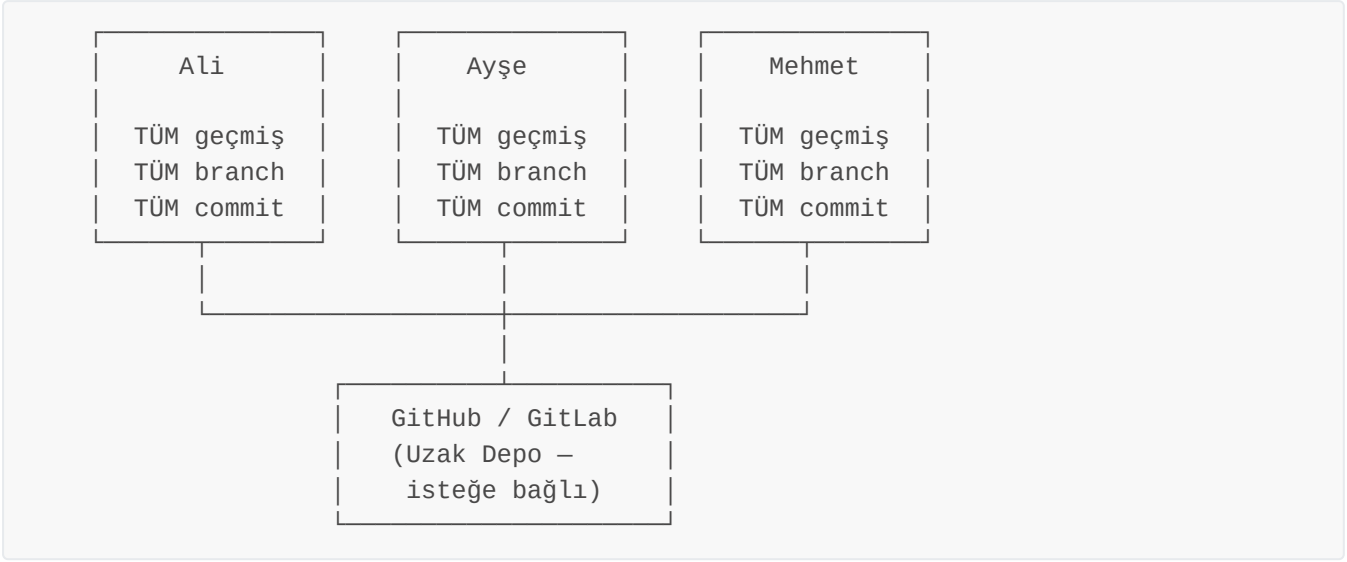
Merkezi (Centralized) Model — SVN Tarzı





- Sunucu = tek gerçek kaynak (single source of truth)
- Geliştiricilerde sadece dosyaların son hali var
- Geçmişe bakmak, branch açmak için → sunucuya bağlanma gerek
- Sunucu çökerse tüm geçmiş kaybolabilir

Dağıtık (Distributed) Model — Git Tarzı



- Her geliştirici, projenin **tam kopyasına** sahip
- İnternetsiz çalışabilirsin: commit at, branch aç, geçmişe bak
- GitHub/GitLab var ama "merkez" değil — sadece kolaylık sağlayan bir paylaşım noktası
- Sunucu çökerse? Yedekten geri alınabilir.

Fark Tablosu

Özellik	Merkezi (SVN)	Dağıtık (Git)
Offline çalışma	❌ Hayır	✅ Tam destek
Hız	🐌 Ağa bağlı	⚡ Yerel, çok hızlı
Yedekleme	😰 Tek nokta	😎 Her kopya bir yedek
Branch açma	🐌 Yavaş, zahmetli	⚡ Saniyenin altında
Geçmişe bakma	🐌 Sunucuya bağlan	⚡ Hep yerel

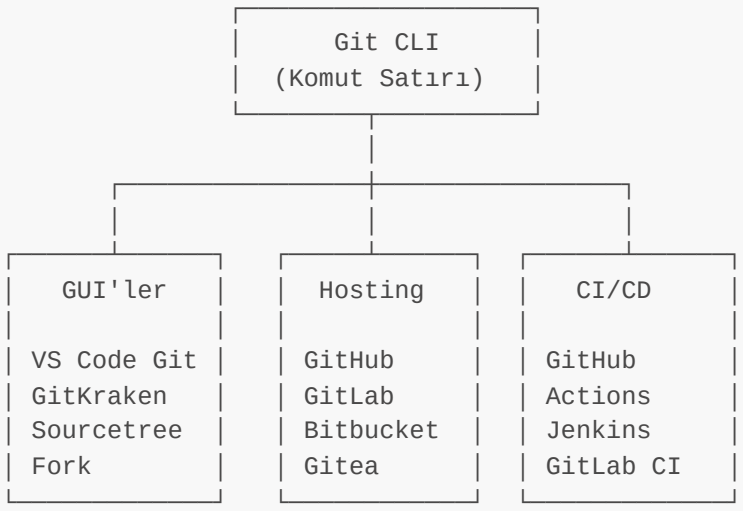
Özellik	Merkezi (SVN)	Dağıtık (Git)
Öğrenme eğrisi	📉 Daha basit	📈 Daha dik ama daha güçlü

Git (Global Information Tracker) Nedir?

⬆ Başa Dön

Git, yazılım geliştirme süreçlerinde kod değişikliklerini zaman içinde kaydetmek ve takip etmek için kullanılan dağıtık bir versiyon kontrol sistemidir. Aynı proje üzerinde çalışan birden fazla yazılımcının kodları birbirine karıştırmadan, eş zamanlı ve düzenli bir şekilde geliştirmesini sağlar. Hatalı güncellemelerde projenin eski sürümlerine kolayca geri dönülmesine imkan tanıyarak veri kaybını önler. Git, her commit'te projenin tamamının anlık görüntüsünü (snapshot) saklar.

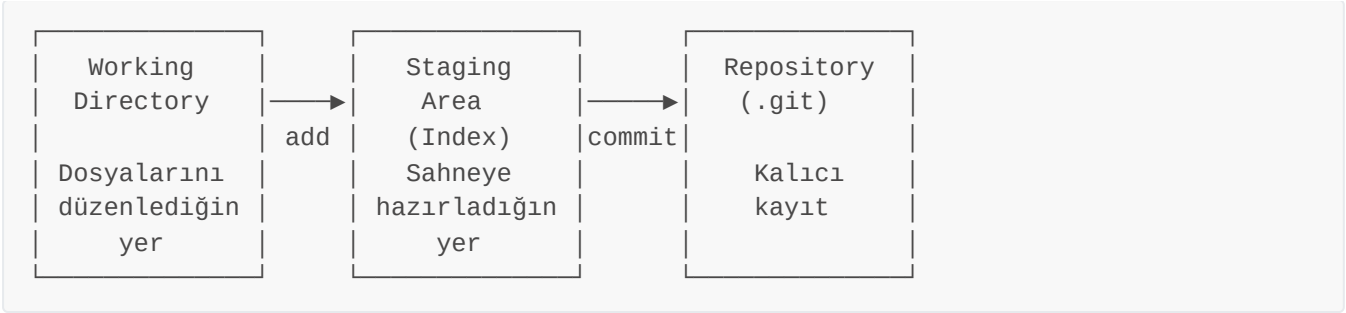
Git Ekosistemi



- **GUI Araçları:** Komut satırına alternatif olarak grafik arayüzler var.
- **Hosting:** Kodunu bulutta tutmak, paylaşmak, işbirliği yapmak için.
- **CI/CD:** Kodun her push'ta otomatik test edilmesi, derlenmesi, deploy edilmesi.

Üç Alan (Three States): Working Directory, Staging Area, Repository

Git, dosyalarını üç temel alanda yönetir.



1. Working Directory (Çalışma Dizini)

Dosyalarını düzenlediğin, kodlarını yazdığın yer. Git'in gözünde bu alan "kontrol dışı" — yani burada ne yaparsan yap, henüz kayıt altına alınmadı.

```
Working Directory'deki dosya durumları:

┌─────────────────┐
│  Untracked      │  Git bu dosyayı hiç bilmiyor (yeni eklendi)
├─────────────────┤
│  Modified       │  Git tanıyor ama son commit'ten beri değişmiş
├─────────────────┤
│  Unmodified     │  Git tanıyor ve değişmemiş (commit'teki haliyle aynı)
└─────────────────┘
```

Dosya Durumları:

- Untracked (İzlenmeyen): Git bu dosyayı hiç bilmiyor. Yeni oluşturulmuş, hiç `git add` edilmemiş.

```
$ touch yeni_dosya.txt
$ git status
Untracked files:
        yeni_dosya.txt
```

- Staged (Hazırlanmış): `git add` ile staging area'ya alınmış. Bir sonraki commit'e dahil edilecek.

```
$ git add yeni_dosya.txt
$ git status
Changes to be committed:
        new file:   yeni_dosya.txt
```

- Unmodified (Değişmemiş): Commit'lenmiş ve o zamandan beri dokunulmamış.

```
$ git commit -m "Yeni dosya eklendi"
$ git status
nothing to commit, working tree clean
# Tüm dosyalar "unmodified" durumunda
```

- Modified (Değişmiş): Commit'lenmiş ama sonra tekrar düzenlenmiş. Git farkı biliyor ama henüz staging'e alınmamış.

```
$ echo "yeni satır" >> yeni_dosya.txt
$ git status
Changes not staged for commit:
        modified:   yeni_dosya.txt
```

2. Staging Area (Hazırlık Alanı / Index)

Staging area, "bir sonraki commit'e neleri dahil edeceğim?" sorusunun cevabı.

```
git add dosya.txt      # dosya.txt'yi sepete koy (staging'e al)
git commit             # sepetteki her şeyi satın al (commit'le)
```

Neden böyle bir ara katman var?

Çünkü her değişikliği aynı commit'e koymak istemezsin. Diyelim ki bir dosyada hem bug fix yaptın hem yeni özellik ekledin. Bunları ayrı commit'lemek isteyebilirsin:

```
# Bug fix'i staging'e al
$ git add login.js

# Commit 1: Sadece bug fix
$ git commit -m "fix: Login hata mesajı düzeltildi"

# Yeni özelliği staging'e al
$ git add dashboard.js

# Commit 2: Sadece yeni özellik
$ git commit -m "feat: Dashboard widget eklendi"
```

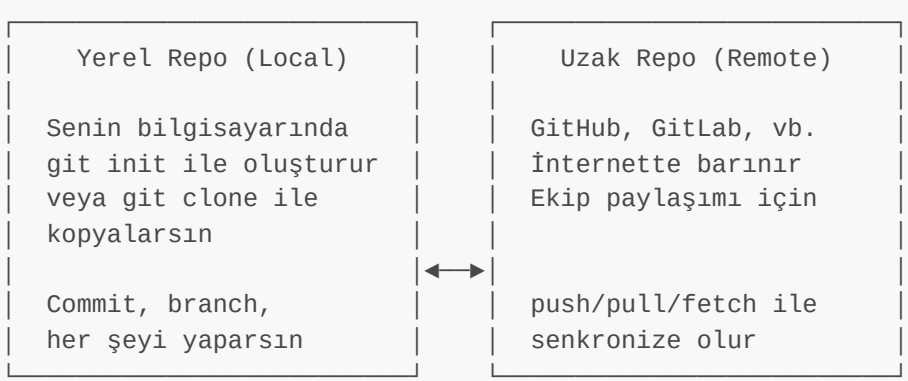
Bu sayede commit geçmişin temiz, anlaşılır ve geri alınabilir olur.

3. Repository (.git dizini)

Repository (kısaca repo), git tarafından izlenen bir proje klasörüdür. Normal bir klasörden farkı, içinde `.git` adlı gizli bir dizin barındırmasıdır.

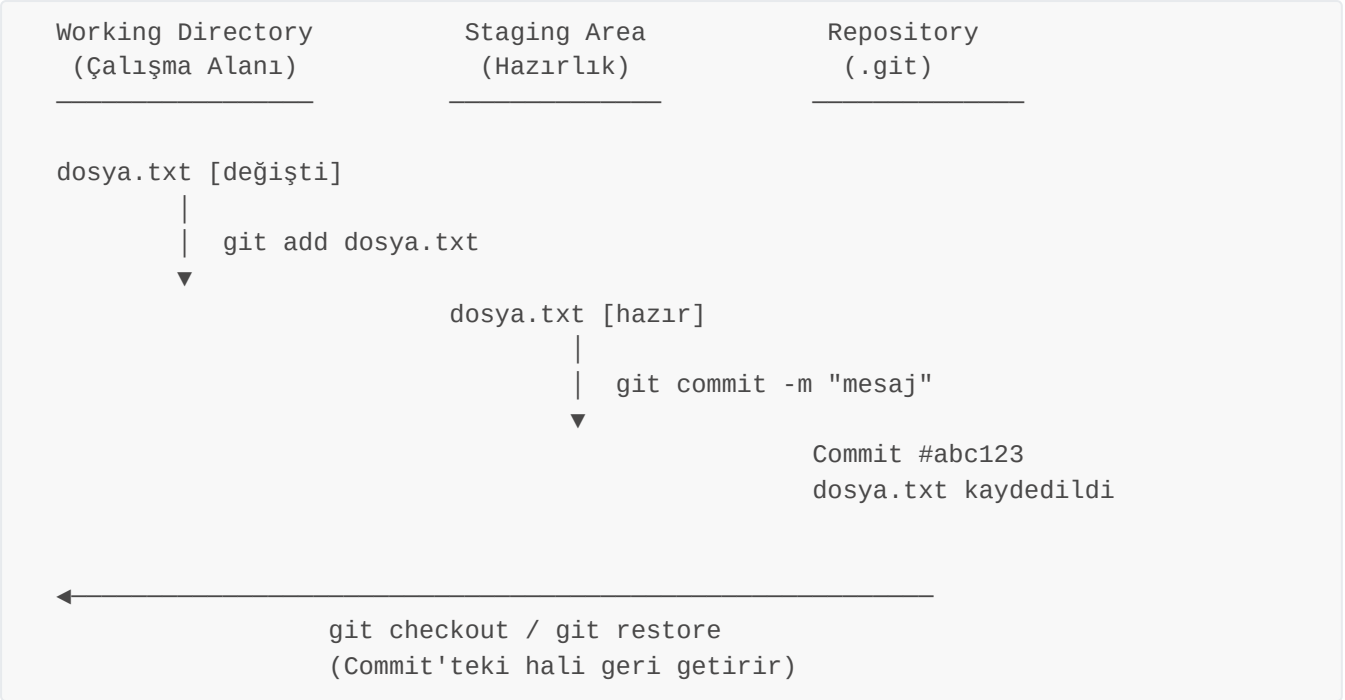
`.git` dizini = Tüm commit geçmişi, branch'ler, ayarlar bu dizinde saklanır. Bu dizini silersen, git geçmişi kaybolur (dosyalar kalır ama geçmiş gider).

İki tür repository vardır:



Commit'lediğin her şey burada **kalıcı olarak** saklanır. Her commit bir snapshot gibi ve bu snapshot'lar zincir gibi birbirine bağlı.

Üç Alanın Akışı



Bu akışın CLI komutları:

```
# 1. Dosya oluştur (Working Directory'de)
$ echo "Merhaba Dünya" > merhaba.txt

# 2. Git'in durumuna bak
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        merhaba.txt

# "Untracked" = Git bu dosyayı tanımıyor, izlemiyor

# 3. Staging'e al
$ git add merhaba.txt

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   merhaba.txt

# "Changes to be committed" = Staging'de, commit'e hazır

# 4. Commit'le
$ git commit -m "İlk dosya eklendi"
[main abc1234] İlk dosya eklendi
```

```
 1 file changed, 1 insertion(+)
 create mode 100644 merhaba.txt

$ git status
On branch main
nothing to commit, working tree clean

# "Clean" = Her şey commit'lenmiş, değişiklik yok
```

Git Yapılandırması | `git config`

Git'in üç farklı yapılandırma seviyesi vardır:

```
┌─────────────────────────────────────────────────────┐
│  System  (/etc/gitconfig)                           │
│  Tüm kullanıcılar, tüm repolar için geçerli         │
│                                                     │
│   ┌─────────────────────────────────────────────┐   │
│   │  Global  (~/.gitconfig)                     │   │
│   │  Bu kullanıcının tüm repoları için geçerli  │   │
│   │                                             │   │
│   │   ┌─────────────────────────────────────┐   │   │
│   │   │  Local  (.git/config)               │   │   │
│   │   │  Sadece bu repo için geçerli        │   │   │
│   │   │  ⬆️ EN YÜKSEK ÖNCELİK               │   │   │
│   │   └─────────────────────────────────────┘   │   │
│   └─────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────┘
```

Git'in sistemine kullanıcıadımızı ve e-mail adresimizi tanımlamak için:

```
git config --global user.name "user_name"
git config --global user.email "user_email"
```

> Not: Depoya özgü yani local kullanıcıadı ve email tanımlama işlemi için `--global` anahtarının belirtilmemesi veya `--global` anahtarı yerine `--local` anahtarı yazılması gerekir. Local ayar, global'den önceliklidir. O repoda yaptığın commit'ler `--local` anahtarı ile tanımladığın kullanıcı bilgileri ile kayıt olur.

⚠️ Dikkat: Bu ayarları yapmazsan, commit atarken seni uyarır veya sistemin hostname ve kullanıcı adını kullanır. (user.name "root" ve user.email "root@localhost" olarak gözükür.)

Tanımlamış olduğumuz kullanıcı adını veya email adresini görüntülemek için:

```
git config --global user.name
git config --global user.email

# Tüm ayarlar için
git config --global --list
```

> Not: Depoya özgü yani local konfigurasyonlar için `git config user.name` veya `git config user.email` komutları, tüm ayarlar için `git config --local --list` komutu kullanılır.

Varsayılan Editör

- VS Code ayarlamak için:git config --global user.name

  `git config --global core.editor "code --wait"`
- Nano ayarlamak için:

  `git config --global core.editor "nano -w"`
- Vim ayarlamak için:

  `git config --global core.editor "vim"`
- Gedit (linux) ayarlamak için:

  `git config --global core.editor "gedit --wait --new-window"`
- Emacs ayarlamak için:

  `git config --global core.editor emacs`
- Sublime Text ayarlamak için:

  `git config --global core.editor "subl -n -w"`

Git varsayılan editörü görüntülemek için:

```
git config --global core.editor
```

> 💡 İpucu: Git, editörü açar ve senin düzenleme yapıp kaydetmeni bekler. `--wait` bayrağı olmadan git hemen devam eder ve boş mesaj alır.

Varsayılan Branch İsmi

Varsayılan brach ismini değiştirmek için:

```
git config --global init.defaultBranch main
```

> 💡 İpucu: Bu ayardan sonra `git init` ile oluşturulan tüm yeni repolar `main` branch'i ile başlar.

Satır Sonu Ayarları (Line Endings)

Windows `CRLF` (\r\n), Linux/Mac `LF` (\n) kullanır. Ekipte karışık işletim sistemi varsa sorun olur.

```
# Windows'ta:
$ git config --global core.autocrlf true
# Checkout: LF → CRLF, Commit: CRLF → LF

# Mac/Linux'ta:
$ git config --global core.autocrlf input
# Checkout: dokunma, Commit: CRLF → LF (yanlışlıkla CRLF girdiyse düzelt)
```

Tüm Ayarları Görüntüleme

```
# Tüm aktif ayarlar (global+local)
$ git config --list
user.name=termux
user.email=community@termux.com
core.editor=code --wait
init.defaultbranch=main
core.autocrlf=true
color.ui=auto

# Belirli bir ayar
$ git config user.name
termux

# Ayarın hangi dosyadan geldiğini görme
$ git config --show-origin user.name
file:/home/user/.gitconfig  termux
```

Ayar Silme

```
# Bir ayarı sil
$ git config --global --unset user.name

# Bir bölümü tamamen sil
$ git config --global --remove-section user
```

Alias (Kısayollar)

Alias'lar, sık kullandığın komutlara kısayol tanımlar. Kısayol tanımlama şekli:

```
$ git config --global alias.<kısayol> <komut_adı>
```

Örnekler:

```
# Kısa log görünümü
$ git config --global alias.lg "log --oneline --graph --all --decorate"

# Durum kısayolu
$ git config --global alias.st "status"

# Kısa commit
$ git config --global alias.cm "commit -m"

# Branch listesi
```

```
$ git config --global alias.br "branch"

# Checkout kısayolu
$ git config --global alias.co "checkout"

# Son commit'i göster
$ git config --global alias.last "log -1 HEAD"

# Unstage (staging'den çıkar)
$ git config --global alias.unstage "restore --staged"
```

Kullanımı:

```
# Artık komutu uzun yazmak yerine:
$ git log --oneline --graph --all --decorate

# Kısa yazabilirsin:
$ git lg
* a1b2c3d (HEAD -> main) Feature X eklendi
* d4e5f6g Login sayfası
* h7i8j9k İlk commit
```

İlk Repository

Adım 1: Repository Oluşturma | `git init`

Bulunduğun dizinde boş bir git deposu oluşturmak için:

```
# Proje klasörü oluştur
$ mkdir web-projem
$ cd web-projem

# Git repository'si başlat
$ git init
Initialized empty Git repository in /home/kullanıcı/web-projem/.git/
```

> ⚠️ Dikkat: `git init` komutu mevcut dosyaları silmez veya değiştirmez. Zaten dosyaları olan bir klasörde de çalıştırabilirsin. Sadece `.git` gizli dizinini ekler. Bu yüzden güvenle kullanabilirsin.

Adım 2: Durum Kontrolü | `git status`

0luşturmuş olduğumuz deponun güncel durum bilgisini görüntülemek için **`git status`** komutunu kullanabilirsiniz. Bu komut ile hangi branch'te olduğumuz veya hangi dosyaların staging alanında (index bölgesi) olduğu gibi bilgiler verir.

Daha kompakt bir çıktı istersen:

```
$ git status -s
?? app.js
?? index.html
?? style.css
```

- `??` = untracked
- `M` = modified
- `A` = added
- `D` = deleted

Adım 3: Staging'e Alma | `git add`

Dosyaları commit'lemeden önce staging area'ya almamız gerekiyor. Sepete koymak gibi.

Tek bir dosyayı staging alanına eklemek için:

```
$ git add <file_name>

# Veya hepsini bir seferde
$ git add .
# Nokta (.) = "bu klasördeki tüm değişiklikleri ekle"

$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   app.js
        new file:   index.html
        new file:   style.css
```

> Not: Üç dosya da staging'de, commit'e hazır.

`git add` Varyasyonları

```
git add dosya.txt               # Tek dosya
git add dosya1.txt dosya2.txt   # Birden fazla dosya
git add .                       # Tüm değişiklikler (bu klasör ve alt)
git add -A                      # Tüm değişiklikler (tüm repo)
git add *.css                   # Pattern ile (tüm CSS dosyaları)
git add src/                    # Klasör ve içindekiler
```

- Dosyayı staging alanından çıkarmak için `git restore --staged <file_name>` komutu kullanılır.
- Takip edilen daha önce commit'lenmiş bir dosyayı staging'den çıkarmak için `git rm --cached <file_name>`komutu kullanılır.
- Dosya staging'den çıkarılır ama silinmez.

`git add .` vs `git add -A` — Farkı

```
# Proje yapısı:
my-project/
├── src/
│   ├── app.js (modified)
│   └── utils.js (new)
├── README.md (deleted)
```

```
└── test.js (modified)

# src/ dizinindeyken:
$ cd src/
$ git add .
# Sadece src/ altındaki değişiklikleri ekler
# README.md'nin silinmesi EKLENMEMİŞ olabilir (eski Git versiyonlarında)

# Herhangi bir yerden:
$ git add -A
# Tüm repo genelindeki tüm değişiklikleri ekler
# Silme, ekleme, düzenleme — hepsi
```

Not: Modern Git versiyonlarında (2.x+) `git add .` da silinmeleri ekler. Ama alışkanlık olarak `-A` daha güvenlidir.

Adım 4: İlk Commit | `git commit`

Staging'deki dosyaları kalıcı olarak kaydetmk için commit gerekli:

```
# Commit oluştur
$ git commit -m "İlk commit: Proje yapısı oluşturuldu"
[main (root-commit) a1b2c3d] İlk commit: Proje yapısı oluşturuldu
 3 files changed, 24 insertions(+)
 create mode 100644 app.js
 create mode 100644 index.html
 create mode 100644 style.css
```

- main — hangi branch'te
- (root-commit) — ilk commit
- a1b2c3d — commit hash'inin kısa hali
- 3 files changed, 24 insertions — 3 dosya eklendi, toplam 24 satır

```
$ git status
On branch main
nothing to commit, working tree clean
```

Clean" — tüm değişiklikler commit'lendi. Temiz bir sayfa.

Not: `-m` ile tek satırlık mesaj yazarsın. Bu kısa değişiklikler için idealdir:

```
git commit -m "Başlık rengi güncellendi"
```

Uzun mesaj yazmak istersen, `-m` olmadan çalıştır:

```
$ git commit
# Editör açılır (VS Code, Nano, vs.)
# İlk satır: Kısa özet (50 karakter)
# Boş satır
# Detaylı açıklama (72 karakter genişlik)

# Editör'de yazacağın mesaj:
```

```
# Kullanıcı giriş formunu ekle

# - Email ve şifre alanları eklendi
# - Form validasyonu yapıldı
# - Hata mesajları Türkçeleştirildi
# - Responsive tasarım uygulandı
```

Adım 5: Değişiklik Yapma ve Yeni Commit

`index.html` ve `style.css` dosyalarına bazı eklemeler yapıp durum kontrolü:

```
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html
        modified:   style.css

no changes added to commit (use "git add" and/or "git commit -a")
```

İki dosya **modified** — değişmiş ama staging'e alınmamış.

```
# Değişiklikleri staging'e al ve commit'le
$ git add .
$ git commit -m "Navigasyon menüsü eklendi"
[main b2c3d4e] Navigasyon menüsü eklendi
 2 files changed, 15 insertions(+), 1 deletion(-)
```

> - `git commit -am "commit mesajı"` : Git add yapmadan direk commit etme işlemi için kullanabilirsiniz.
> - `git commit --amend -m "yeni commit mesajı"` : En son yapılan commit mesajını değiştirmek için kullanılır.

⚠️ **Dikkat:** `git commit -a` sadece **zaten izlenen** dosyaları otomatik ekler. **Yeni oluşturulan** (untracked) dosyaları eklemez! Yeni dosya varsa önce `git add` yapmalısın.

Adım 6: Geçmişi İnceleme | `git log`

Yapılan commitleri görüntülemek için:

```
$ git log
commit b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1 (HEAD -> main)
Author: kullanıcı <email@adresiniz.com>
Date:   Mon Mar 1 10:30:00 2026 +0300

    Navigasyon menüsü eklendi

commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
Author: kullanıcı <email@adresiniz.com>
Date:   Mon Mar 1 10:00:00 2026 +0300

    İlk commit: Proje yapısı oluşturuldu
```

`git log` Varyasyonları

```
# Tek satırlık özet
$ git log --oneline
b2c3d4e (HEAD -> main) Navigasyon menüsü eklendi
a1b2c3d İlk commit: Proje yapısı oluşturuldu

# Grafiksel gösterim (branch'lerle birlikte)
$ git log --oneline --graph --all
* b2c3d4e (HEAD -> main) Navigasyon menüsü eklendi
* a1b2c3d İlk commit: Proje yapısı oluşturuldu

# Son N commit
$ git log -2        # Son 2 commit
$ git log -1        # Sadece son commit

# Hangi dosyalar değişmiş
$ git log --stat
commit b2c3d4e (HEAD -> main)
    Navigasyon menüsü eklendi

 index.html | 10 ++++++++--
 style.css  |  7 +++++++
 2 files changed, 15 insertions(+), 2 deletions(-)

# Değişiklikleri satır satır göster
$ git log -patur
# Her commit'in diff'ini gösterir (çok detaylı)

# Belirli bir dosyanın geçmişi
$ git log -- index.html
# Sadece index.html'i etkileyen commit'ler
```

> Not: `git log --oneline --graph` komutu branch'lerle birlikte çok kullanışlıdır.
>
> `HEAD` : Git'in içinde bulunduğumuz konumu belirten bir referanstır. Genellikle en son commit'i işaret eder. Bu, nerede olduğumuzu ve hangi commit üzerinde çalıştığımızı belirlememizi sağlar.

`.gitignore` — Neyi İzlememeliyiz?

Git'e koymak istemediğiniz proje dosyasını yani izlenmesini istenmeyen dosyalar veya dizinler `.gitignore` dosyasında belirtilerek bu dosyalar görmezden gelmesine izin verilebilinir. Proje kök dizinine eklenir.

```
# .gitignore dosyası oluştur
$ cat > .gitignore << 'EOF'
# Bağımlılıklar
node_modules/
vendor/

# Ortam değişkenleri
.env
.env.local

# IDE dosyaları
.vscode/
.idea/
*.swp
*.swo

# İşletim sistemi dosyaları
.DS_Store
Thumbs.db

# Build çıktıları
dist/
build/
*.min.js
*.min.css

# Log dosyaları
*.log
npm-debug.log*
EOF

# Dizin klasöründeki b dosyası hariç tüm dosyaları kapsar
!dizin/b

$ git add .gitignore
$ git commit -m "chore: .gitignore dosyası eklendi"
[main 4d5e6f7] chore: .gitignore dosyası eklendi
 1 file changed, 23 insertions(+)
```

Belirli bir commit'te ne olduğunu görmek için:

```
$ git show 2b3c4d5
commit 2b3c4d5...
Author: Tolgahan Kaya <tolgahan@example.com>
Date:   Mon Mar 1 10:15:00 2026 +0300

    feat: Hakkımda bölümü eklendi

diff --git a/index.html b/index.html
```

```
--- a/index.html
+++ b/index.html
@@ -13,5 +13,9 @@
     </header>
+    <section id="about">
+        <h2>Hakkımda</h2>
+        <p>5 yıllık deneyime sahip bir yazılım geliştiriciyim.</p>
+    </section>
     <script src="js/main.js"></script>

# En son commit'i görmek için:
$ git show HEAD
# veya
$ git show
# (parametre olmadan HEAD'i gösterir)
```

Özet

- `git init` ile boş bir repository oluşturursun — `.git` dizini eklenir, dosyalara dokunulmaz
- `git status` ile projenin durumunu kontrol edersin — bu en sık kullanacağın komut
- `git add` ile değişiklikleri staging area'ya alırsın — commit'e hazırlarsın
- `git commit -m "mesaj"` ile staging'dekileri kalıcı olarak kaydedersin — projenin fotoğrafını çekersin
- `git log` ile commit geçmişini incelersin — `--oneline` ve `--graph` en faydalı bayraklar
- `.gitignore` ile izlenmemesi gereken dosyaları belirlersin — `node_modules`, `.env`, IDE dosyaları

Dosya İşlemleri	
`git rm <file_name>`	Dosya silmek için kullanılır
`git rm -r <directory_name>/`	Dizin silmek için kullanılır
`git mv <file_name> <new_file_name>`	Dosya adı değiştirmek için kullanılır
`git mv <file_name> <directory_name>/`	Dosyayı taşımak için kullanılır
`git mv <file_name> <directory_name>/<new_file_name>`	Dosyayı adını değiştirerek taşımak için kullanılır

- **Zamanı geri alma yani git deposunda geçmiş tarihli bir commit'e geri gitmemiz için:**

```
git checkout <commit_id>
```

- **Gittiğimiz committen geri gelmemize yani son commite geri dönmek için:**

```
git checkout master || git switch master
```

- **Git versiyon değiştirme (silinen tüm dosyaları geri getirmek) için:**

```
git checkout <commit id> -- .
```

- `git diff` **: Staging alanına eklenmeden önce tüm dosyalarda yapılan değişiklikleri gösterir.**
- `git diff <file_name>` **: Staging alanına eklenmeden önce ismi verilen tek bir dosyada yapılan değişiklikleri gösterir.**
- `git diff --staged` **: Git deposu ile staging alanındaki değişiklikleri gösterir.**

- `git branch` **: Yerelimizde kaç dal (branch) olduğunu ve hangi dalda bulunduğumuzu gösterir.**
- `git branch --all` **: Yerelimizde ve uzak depodaki tüm dalları gösterir.**
- `git branch -r` **: Uzak depodaki dalları gösterir.**
- `git branch <branch_name>` **: Yeni dal (branch) oluşturmak için kullanılır.**
- `git branch -m <branch_name> <new_branch_name>` **: Dal adını değiştirir, ancak yeni isimde bir dal varsa hata verir.**
- `git branch -M <branch_name> <new_branch_name>` **: Dal adını değiştirir, yeni isimde bir dal varsa üzerine yazar (force).**
- `git branch -D <branch_name>` **: Lokalde ismi verilen bir dalı (branch) silmek için kullanılır.**

- `git switch <branch_name>` **: Girilen branch'a geçiş yapar.**
- `git checkout <branch_name>` **: Uzak depodan yerel depoya indirilen branch'a geçiş yapar.**

- `git merge <branch_name>` **: Master branch'ındayken ismi verilen diğer branch'ı master branch'ıyla birleştirmek için kullanılır.**

- `git stash` **: Git versiyon kontrol sistemi kullanılarak yapılan değişiklikleri geçici olarak kaydetmenizi sağlayan bir özelliktir. Bu, henüz tamamlanmayan bir iş üzerinde çalışırken veya bir dal üzerinde çalışırken aniden başka bir acil işle ilgilenmeniz gerektiğinde özellikle kullanışlıdır.**

- `git stash list` **: Kaydedilen tüm stash'leri listeler.**
- `stash@{0}` **: Git stash listesi içerisindeki ilk yani en son eklenen geçici değişiklikler listesindeki kaydedilmiş çalışma dizininin (working directory) saklandığı referans adıdır.**

- `git stash apply` **: En son kaydedilen stash'i geri yükler.**

- `git stash apply stash@{n}` : **Belirtilen numaralı stash'i geri yükler.**

- `git stash drop` : **En son kaydedilen stash'i siler.**
- `git stash drop stash@{n}` : **Belirtilen numaralı stash'i siler.**

- `git stash pop` : **Komutu, en son kaydedilen stash girdisini alır ve bu değişiklikleri uygular (apply) ve stash havuzundan (stash pool) kaldırır. Yani, pop işlemi stash havuzundan en son eklenen stash girdisini çıkarır ve çalışma dizinindeki değişiklikleri bu girdiye göre günceller.**

- `git stash clear` : **Komutu ise stash havuzundaki tüm stash girdilerini siler.**

- `git reset <commit_id>` : **Belirtilen bir commit'e geri dönmeyi sağlar ve bu işlem esnasında commit'ler silinir değişiklikler kalır.**
- `git reset --hard <commit_id>` : **Belirtilen bir commit'e geri dönmeyi sağlar ve bu işlem esnasında commit'ler ve değişiklikler silinir.**

- `git revert <commit_id>` : **Belirli bir commit'i geri alırsınız ve bu işlem sonucunda yeni bir commit oluşur. Bu sayede, Git geçmişi değiştirilmez, ancak istenmeyen değişiklikler geri alınmış olur.**

- `git rebase` **komutu, git'te branch (dal) yönetiminde kullanılır ve commit geçmişini düzenlemeye yarar. Temel amacı, bir branch'in temelini başka bir branch'in en son haliyle değiştirmek ve daha "düzgün" bir commit geçmişi oluşturmaktır.**

  Yani:

  - **Branch'inizdeki commitleri, hedef branch'in en güncel commitlerinin üstüne "taşır".**
  - **Genellikle** `git merge` **ile benzer problemlere çözüm getirir ama geçmişi daha lineer hale getirir.**

  En yaygın kullanım:

  ```
  git checkout feature-branch
  git rebase main
  ```

  Bu komut, `feature-branch` **dalındaki değişiklikleri,** `main` **dalının en güncel halinin üzerine taşır.**

  Faydaları:

  - **Commit geçmişi daha temiz ve düz olur.**
  - **Ortak bir temel üzerinde çalışmayı kolaylaştırır.**
  - **Merge commitleri oluşturmaz (tüm commitler tek bir çizgide görünür).**

  Dikkat edilmesi gereken noktalar:

  - **Rebase işlemi, var olan commitleri yeniden yazdığı için, paylaşılan branch'lerde kullanırken dikkatli olunmalıdır.**
  - **Başkaları tarafından erişilen branch'lerde rebase yapılmamalı.**

  Kısaca, `git rebase` **, branch'leri birleştirirken temiz ve düzenli bir commit geçmişi sağlar.**

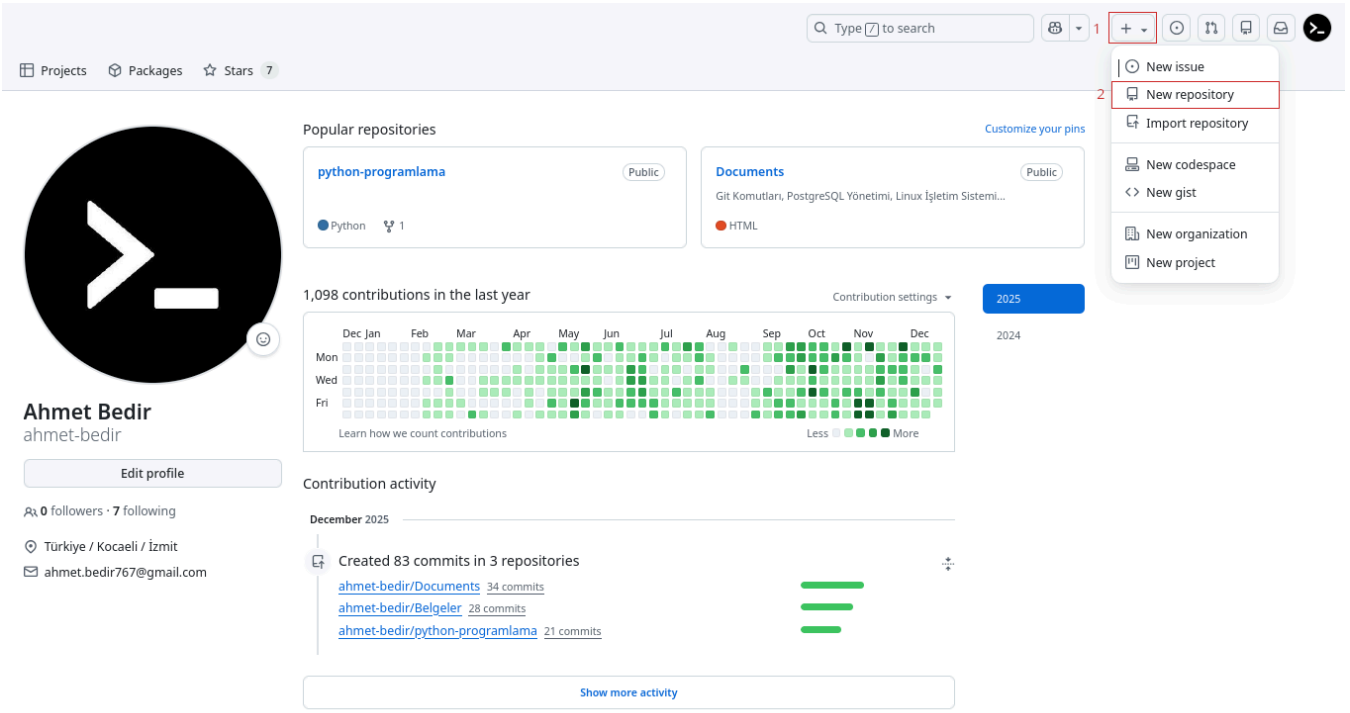
Remote

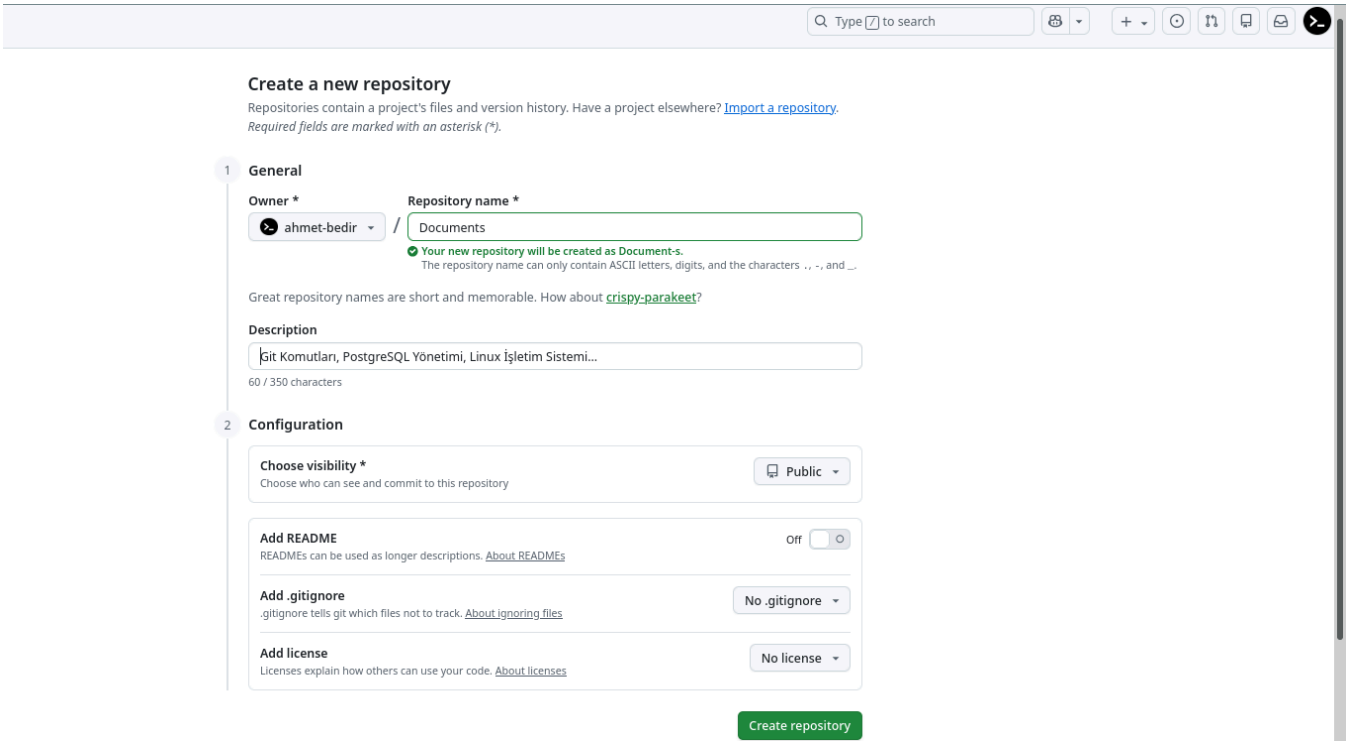
Remote uzun linkleri kısaltmamıza ve onları bir isim ile bağdaştırmamızı sağlar. Örneğin: `git remote add <remote_name> https://github.com/<github_username>/<repo_name>.git` **komutunda** `<remote_name>` **kısmına istediğiniz ismi verebilirsiniz. Yani** `<remote_name>` **dediğimizde** `https://github.com/<github_username>/<repo_name>.git` **bu url temsil edilmektedir.**

GitHub'da yeni bir repo (depo) oluşturmak için aşağıdaki adımları takip edebilirsiniz:

1. **GitHub Hesabınıza Giriş Yapın:**
   https://github.com adresine gidin ve hesabınıza giriş yapın.
2. **Yeni Depo Oluşturma Sayfasına Gidin:**
   Sağ üst köşedeki "+" simgesine tıklayın ve açılan menüden "New repository" (Yeni depo) seçeneğini seçin.
   Alternatif: https://github.com/new adresine doğrudan gidebilirsiniz.
3. **Depo Bilgilerini Doldurun:**
   - **Owner (Sahip):** Depoyu kendi hesabınızda mı yoksa bir organizasyonda mı oluşturmak istediğinizi seçin.
   - **Repository Name (Depo Adı):** Depoya benzersiz bir isim verin.
   - **Description (Açıklama):** (İsteğe bağlı) Depo hakkında kısa bir açıklama yazın.
   - Visibility (Görünürlük):
     - **Public:** Herkes görebilir.
     - **Private:** Sadece siz ve davet ettikleriniz görebilir.
4. **İlk Ayarları Yapın (Opsiyonel):**
   - "Add a README file" kutucuğunu işaretleyerek başlangıçta bir README dosyası oluşturabilirsiniz.
   - .gitignore ve lisans eklemek için uygun seçenekleri belirleyebilirsiniz.
5. **"Create Repository" Butonuna Tıklayın:**
   Sayfanın en altında yeşil renkli "Create repository" butonuna tıklayın.
6. **(Opsiyonel) Depoyu Bilgisayarınıza Klonlayın:**
   Oluşturduğunuz repoyu kendi bilgisayarınıza kopyalamak için "Code" butonuna tıklayın ve bağlantıyı kopyalayıp terminalde şu komutu kullanın:

   ```
   git clone https://github.com/<github_username>/<repo_name>.git
   ```





- **Yeraldeki repoya uzak sunucudaki repoyu ilişkilendirmek için uzak repo adresini https protokolü ile ekliyoruz.**

```
git remote add <remote_name> https://github.com/<github_username>/<repo_name>.git
```

PAT (Personal Access Token) Nedir, Ne İçin Kullanılır?

PAT (Personal Access Token) GitHub, GitLab gibi platformlarda kullanıcıların hesabına komut satırı (CLI) veya üçüncü parti uygulamalar üzerinden erişim sağlamak için kullandığı bir kimlik doğrulama yöntemidir. Özellikle şifre ile kimlik doğrulama devre dışı bırakıldığından, HTTPS ile erişimlerde şifre yerine PAT kullanılır.

Kullanım amaçları:

- **Komut satırı veya uygulamalar üzerinden kimlik doğrulama yapmak**
- **Otomasyon scriptlerinde veya CI/CD süreçlerinde erişim sağlamak**
- **Klasik şifre ile giriş yerine, daha güvenli ve süreli erişim belirteçleri üretmek**

PAT Nasıl Oluşturulur?

1. GitHub hesabına giriş yap
Sağ üstteki profil fotoğrafına tıkla → *Settings* menüsüne gir.

2. Developer settings → Personal access tokens başlığına gel.

3. ***Tokens (classic)*** **veya** ***Fine-grained tokens*** **sekmesini seç.**

4. Generate new token butonuna tıkla.

5. Açılan formda:

- **Token'a bir isim ver (ör: "CLI erişimi için")**
- **Erişim süresi (expiration) belirle**
- **Hangi izinlere sahip olacağını seç (repo, workflow, user, vs.)**

6. Generate token diyerek token'ı oluştur.
Oluşan token'ı güvenli bir yere kopyala (çünkü bir daha gösterilmez).

Komut Satırında PAT ile Kimlik Doğrulama

> - **PAT'i kullanarak HTTPS ile push/pull işlemi yapmak için:**

```
git remote set-url <remote_name> https://<github_username>:
<pat>@github.com/<github_username>/<repo_name>.git
```

Not : Kısacası, PAT güvenli ve modern bir kimlik doğrulama yöntemidir ve GitHub gibi platformlarda şifreyle girişin yerini almıştır.

SSH Key Oluşturma ve Yapılandırma

SSH Neden Gerekli?

GitHub, GitLab gibi platformlara kodu göndermek (push) için kimliğini doğrulamalısın. İki yol var:

1. **HTTPS:** Her seferinde kullanıcı adı/şifre veya token girersin.
2. **SSH:** Bir kere anahtar çifti oluşturursun, sonra şifresiz bağlanırsın.

SSH Nasıl Çalışır?

```
┌──────────────────────────────────────────────────┐
│                  SSH Key Çifti                   │
│                                                  │
│  🔑 Private Key (Özel Anahtar)                   │
│  ~/.ssh/id_ed25519                               │
│  ❌ KİMSEYLE PAYLAŞMA! Bilgisayarında kalır.     │
│                                                  │
│  🔓 Public Key (Genel Anahtar)                   │
│  ~/.ssh/id_ed25519.pub                           │
│  ✅ GitHub'a, GitLab'a ekle. Paylaşılabilir.     │
│                                                  │
│  Not: Kapının kilidi (public) herkeste olabilir  │
│  ama anahtarı (private) sadece sende.            │
└──────────────────────────────────────────────────┘
```

- **Public key** = açık asma kilit — GitHub'a veriyorsun, "benim mesajlarımı bununla kilitle" diyorsun
- **Private key** = anahtarın — sadece sen açabilirsin

Komut Satırında SSH ile Kimlik Doğrulama

1.SSH anahtarınızı oluşturmak için:

```
ssh-keygen -t ed25519 -C "email@adresiniz.com"
```

Not: Ssh key anahtarı ed25519 algoritması ile modern, güvenli, hızlı.

Terminalde size bazı sorular sorar:

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kullanıcı/.ssh/id_ed25519):
# Varsayılan konum için enter'a bas veya yeni bir konum gir

Enter passphrase (empty for no passphrase):
# Güçlü bir parola gir (önerilir) veya boş bırak

Enter same passphrase again:
# Aynı parolayı tekrar gir

Your identification has been saved in /home/kullanıcı/.ssh/id_ed25519
Your public key has been saved in /home/kullanıcı/.ssh/id_ed25519.pub
The key fingerprint is:
```

```
SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx email@adresiniz.com
The key's randomart image is:
+--[ED25519 256]--+
|        .oo.     |
|       . o.o     |
|        = +.o    |
|       . B.=     |
|      . S.=.o    |
|     . + o.+.    |
|      + o.+.     |
|     . +.o.+     |
|      o.oo=.     |
+----[SHA256]-----+
```

2.Genel anahtarı kopyalamak için:

```
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIG... email@adresiniz.com

# çıktının tamamını kopyala
```

3.Genel anahtarı GitHub'a ekleyin:

- GitHub'da sağ üstte profil fotoğrafınıza tıklayın → Settings → SSH and GPG keys → New SSH key
- Title: "Kişisel Laptop" kTitle kısmına bir isim verin)
- Key type: "Authentication Key"
- Key: Kopyaladığın public key'i yapıştır
- Add SSH key butonuna tıkla

4.Bağlantıyı test edin:

```
ssh -T git@github.com
```

İlk seferde "Are you sure you want to continue connecting (yes/no/[fingerprint])?" sorusuna yes yazın. "Hi username! You've successfully authenticated..." mesajı görmelisiniz.

Hata alırsan:

```
# Detaylı debug modu (doğru key kullanılıyor mu?)
$ ssh -vT git@github.com
```

5.Depoyu SSH ile kullanmak için depo bağlantı adresiniz şu şekilde olmalı:

```
git@github.com:<github_username>/<repo_name>.git
```

6.Var olan bir depoyu HTTPS'i SSH'ye çevirmek için:

```
git remote set-url <remote_name> git@github.com:<github_username>/<repo_name>.git
```

`git remote -v`: Yerele indirdiğiniz (klonladığınız) bir github deposunun hangi hesaptan veya hangi url üzerinden klonlandığını ve hangi yöntem ile bağlantı kulduğunu öğrenmek için kullanılır.

- **Uzak sunucudaki repoya yapılan commitleri göndermek için kullanabilirsiniz.**

```
git push -u <remote_name> <branch_name>
```

Not : -u (upstream) ifadesi, varsayılan yukarı akış depoya (remote_name) ve ana dal (branch_name) için bir yer işaretçisi belirler. Bu işaretçi sayesinde, bir sonraki git push komutunu çağırdığınızda, Git (remote_name) ve (branch_name) argümanlarını tekrarlamak yerine bu yer işaretçisini kullanarak sadece `git push` komutu ile aynı işlemi yapabilirsiniz.

- **Fetch işlemi, uzak depodaki yeni değişiklikleri lokal depoya indirir ancak lokaldeki çalışma dizinine (working directory) birleştirmez. Bu işlem, uzak depodaki değişikliklerin var olup olmadığını kontrol etmek için kullanır.**

```
git fetch <remote_name> <branch_name>
```

- **Pull işlemi, uzak depodaki yeni değişiklikleri hem indirir hem de lokaldeki değişikliklerle birleştirir. Bu işlem, uzak depodaki değişiklikleri lokaldeki çalışma dizinine (working directory) eklemek istediğinizde kullanılır.**
  git pull = git fetch + git merge

```
git pull <remote_name> <branch_name>
```

- **Bu komut, uzak depodaki tüm dosyaları, tarihçeyi ve yapılandırmayı https protokolü ile kopyalar ve lokalde yeni bir git deposu oluşturur.**

```
git clone https://github.com/<github_username>/<repo_name>.git
```

Not: Bu işlem sonucunda username ve password girişi için username kısmına kullanıcıadı şifre kısmına ise oluşturduğunuz pat girilir. Yalnız bu kullanımda her `push` için giriş gerekir.

Çözüm için klonlama yaparken:

```
git clone https://<username>:<pat>@github.com/<username>/<repo_name>.git
```

- **Bu komut, uzak depodaki tüm dosyaları, tarihçeyi ve yapılandırmayı ssh protokolü ile kopyalar ve lokalde yeni bir git deposu oluşturur.**

```
git clone git@github.com:<github_username>/<repo_name>.git
```